A Hopfield-based Replay Buffer for DQN: An Experimental Study on CartPole-v1

İbrahim Emir Tosun
Erasmus Student
Bio-Inspired Machine Learning (APBIO), Universidad de Vigo

Academic Year 2025–2026

Abstract

This report describes a hybrid reinforcement learning agent that couples a Modern Hopfield Network with Deep Q-Learning (DQN). The Hopfield network replaces the standard uniform-random replay buffer: transitions are sampled according to attention weights conditioned on the most recently observed state, so that past experiences most similar to the current situation are preferentially replayed. The system is tested on the CartPole-v1 environment. An initial experiment with the default retrieval temperature $\beta = 5$ shows that the hybrid underperforms the plain DQN baseline by about 56% in final return. A hyperparameter sweep then reveals that β is by far the most influential parameter: lowering it to $\beta = 1.0$ flips the result, and the best hybrid configuration slightly outperforms the baseline (263.7 ± 82.0 vs. 236.4 ± 98.6, +11.5%). The improvement is not statistically significant ($p = 0.68$, Welch's t-test), which suggests that on a well-posed environment such as CartPole the Hopfield buffer offers only a marginal benefit over random replay. The project's main contribution is showing, with paired 5-seed experiments and three hyperparameter sweeps, that a biologically-inspired replay mechanism can match but does not clearly exceed random replay in this setting, and that its behaviour is strongly governed by the retrieval temperature.

1 Introduction

Deep Q-Networks (DQN) store observed transitions in an experience replay buffer and, at every training step, draw a mini-batch uniformly at random from that buffer [2]. Random sampling decorrelates the transitions seen by the optimiser, which stabilises gradient estimates, but it ignores any notion of *relevance*: a transition recorded in a completely unrelated state is equally likely to be drawn as one that closely resembles the agent's current situation.

A natural question is whether the replay buffer can be made *content-addressable*. Modern Hopfield Networks, recently reformulated by Ramsauer et al. [5], provide exactly such a mechanism: given a query vector, they return a softmax-weighted combination of the stored patterns, with sharpness controlled by an inverse-temperature parameter β. Functionally they are identical to attention heads, but framed as an associative memory with a clear energy interpretation.

This project asks a simple empirical question: if we replace DQN's uniform replay with a Hopfield-based replay buffer that retrieves transitions similar to the most recent observation, does the agent learn faster, more robustly, or to a higher final return? The four families of bio-inspired methods covered in the course (evolutionary, swarm, immune-inspired, neuro-inspired) all provide candidate components; I picked a neuro-inspired one because its mathematics matches modern attention mechanisms closely and integrates cleanly with a standard deep RL pipeline.

Contributions. This report contributes (i) a concrete implementation of `HopfieldReplayBuffer` compatible with Stable-Baselines3's DQN; (ii) a paired 5-seed comparison of hybrid and baseline on CartPole-v1; (iii) three hyperparameter sweeps (one bio-inspired, one structural, one RL) that

identify the retrieval temperature β as the only parameter with a large effect on performance; and (iv) a follow-up experiment with the best-found configuration that closes and slightly reverses the initial gap.

Outline. Section 2 reviews DQN replay buffers and Modern Hopfield Networks. Section 3 describes the hybrid architecture and the integration with Stable-Baselines3. Section 4 reports the main experiment, the sweeps, and the best-configuration run. Section 5 addresses the ten evaluation questions from the project specification. Section 6 concludes.

2 Background

2.1 Deep Q-Learning and experience replay

DQN learns a parametric action-value function $Q_\theta(s, a)$ by minimising the temporal-difference error

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{B}}\left[\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a)\right)^2\right], \tag{1}$$

where $\mathcal{B}$ is a replay buffer of past transitions and $\bar{\theta}$ are the (slowly updated) target-network parameters. Sampling from $\mathcal{B}$ at random breaks the temporal correlation between consecutive states, which is what makes function approximation with a neural network stable. Prioritised Experience Replay [6] biases sampling by TD-error and was an early demonstration that non-uniform replay can help; our Hopfield-based scheme is a different kind of prioritisation, biased by state similarity rather than by error.

2.2 Modern Hopfield Networks

A classical Hopfield network stores N binary patterns $\{\xi_i\}_{i=1}^N$ and retrieves one of them given a noisy cue by iterative energy minimisation. Its storage capacity is linear in the pattern dimension, which severely limits its use as a general-purpose memory. Ramsauer et al. [5] introduced a continuous variant in which retrieval is a single matrix operation

$$\text{retrieve}(q) = X^\top \,\text{softmax}(\beta X q), \tag{2}$$

where $X \in \mathbb{R}^{N\times d}$ is the matrix of stored patterns (one per row), $q \in \mathbb{R}^d$ is the query, and $\beta > 0$ is an inverse temperature. This formula is identical to a standard attention layer with X acting as both keys and values. The storage capacity grows exponentially with d, and the associated energy function is

$$E(q) = -\tfrac{1}{\beta}\log\textstyle\sum_i \exp(\beta x_i^\top q) + \tfrac{1}{2}\|q\|^2, \tag{3}$$

which lets us monitor how "confident" the memory is about a given query.

2.3 Why combine them?

Uniform replay is a very cheap form of prioritisation: it weights every past transition equally. A Hopfield memory, at the opposite extreme, can concentrate all weight on a single stored pattern when β is large. Intermediate values of β interpolate between the two. The working hypothesis of this project is that a well-tuned Hopfield replay buffer lies in a sweet spot between "fully random" and "fully content-addressed" and should therefore at least match, and perhaps slightly improve on, plain DQN on a simple environment.

There is a second, subtler motivation. In DQN the target $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$ is itself a bootstrapped estimate, and errors in it propagate through the Bellman update. If the mini-batch is dominated by transitions whose states are close to the current one, the regression becomes locally specialised and the network may fit the short-term Q-landscape well at the cost of its global accuracy. Random sampling averages over the whole replay buffer and implicitly regularises against

this local overfitting. A Hopfield retrieval with a moderate β is therefore only expected to help when "relevance" is more informative than "coverage" — when the problem has clearly separated regions or rare-but-important transitions. CartPole-v1 has neither property: it is a low-dimensional, dense-reward task in which almost every transition carries useful information. This is a deliberate choice on my part: if the hybrid fails even on an environment this benign, it is a clear signal that the design is not a general improvement over random replay.

3 Methodology

3.1 Environment and base RL algorithm

I use the `CartPole-v1` environment from Gymnasium [1]: a 4-dimensional continuous state, two discrete actions (push left or right), reward of +1 per step, and a maximum return of 500. The base RL algorithm is DQN as implemented in Stable-Baselines3 version 2.3 [4]. The hyperparameters used throughout are the Stable-Baselines3 RL Zoo defaults for CartPole-v1 [3]: learning rate 2.3×10^{-3}, two hidden layers of 256 units, train_freq = 256 steps, gradient_steps = 128, target update every 10 training steps, exploration fraction 0.16, and a buffer of 100 000 transitions. I train for 50 000 environment steps per run, which is enough for the baseline to reach a final return of several hundred on most seeds but not enough to always saturate at 500; this leaves headroom for the hybrid to improve on the baseline.

3.2 Hopfield replay buffer

The Hopfield memory is a small PyTorch module with a pattern matrix $X \in \mathbb{R}^{N \times d}$, an inverse temperature β, and three operations:

- `store(x)`: overwrite the oldest patterns if the memory is full (ring-buffer); otherwise append.
- `retrieve_weights(q)`: return $\mathrm{softmax}(\beta X q)$.
- `mean_energy(q)`: return the energy associated with the query, used as a diagnostic.

`HopfieldReplayBuffer` subclasses `stable_baselines3.common.buffers.ReplayBuffer`. Its contract is:

1. On `add()`, the transition is appended to the parent class's arrays *and* the observation is appended to the Hopfield memory. The most recent observation is cached in `self._last_obs`.
2. On `sample(batch_size)`, the buffer computes the attention weights of `self._last_obs` over all currently stored patterns, uses those weights to draw `batch_size` indices with replacement, and delegates the actual tensor construction to the parent class's `_get_samples()`.

Pseudocode for the sampling step:

```
def sample(self, batch_size, env=None):
    w, n = self.hopfield.retrieve_weights(self._last_obs)
    w = w.squeeze(0).cpu().numpy()
    w = w[:min(n, self.size())]
    w = w / w.sum()
    idx = np.random.choice(len(w), size=batch_size, replace=True, p=w)
    return self._get_samples(idx, env=env)
```

3.3 Architecture overview

Figure 1 shows how the components interact. The environment produces transitions in the usual way, and the DQN policy is unchanged. The only difference is that the bridge between the environment's transition stream and the DQN training loop goes through the Hopfield memory rather than a flat array.

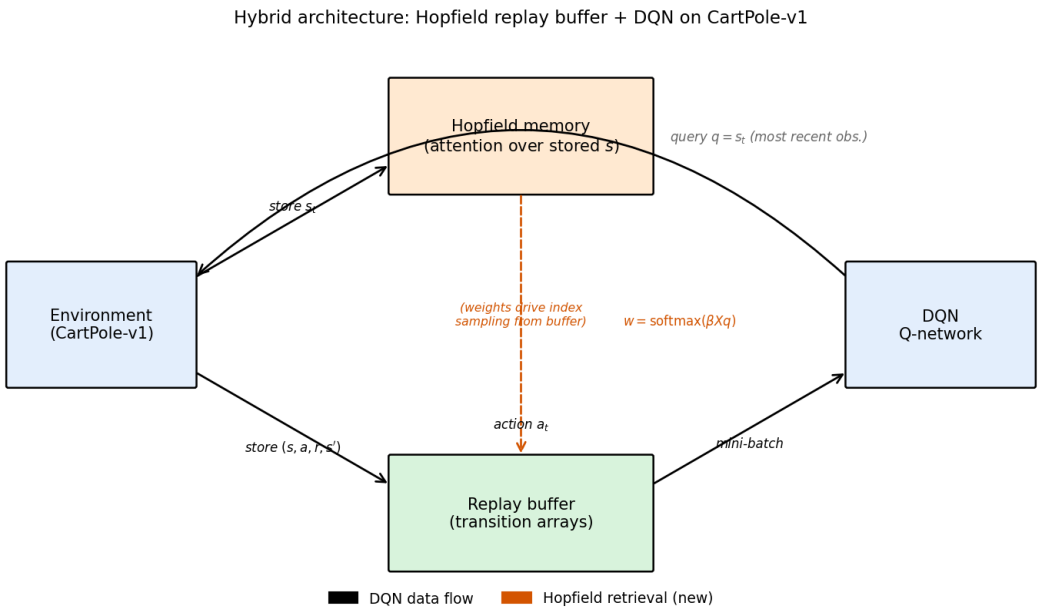

Figure 1: Architecture of the hybrid system. Solid arrows denote the normal DQN data flow; dashed arrows highlight the changes introduced by the Hopfield replay buffer. The only point of integration is the replay buffer itself.

3.4 Validity of the coupling

The project specification gives three validity checks for a hybrid design:

Is the coupling non-trivial? Yes. Removing the Hopfield component and falling back to the parent class's random sampling recovers plain DQN exactly; the two systems differ in what `sample()` returns.

Does the bio-inspired component have adaptive state? Yes. The pattern matrix X grows and is rewritten as new transitions arrive, so the set of retrievable patterns evolves continuously during training.

Can the data flow be drawn? Yes, see Figure 1. The bridge is localised to a single module; no other part of DQN is modified.

3.5 Experimental protocol

All reported numbers are averages over 5 independent seeds for the main experiment and 3 seeds per configuration for the sweeps. Each run is a full 50 000-step training on a fresh environment and a freshly initialised network, with deterministic seeds set in Python, NumPy, PyTorch, and the Gymnasium environment. Raw per-episode CSV logs are written to disk as each run completes, so experiments are restartable.

The 5-seed budget is reserved for the headline claims (main experiment and best-configuration validation), in line with the project specification's requirement that "all reported results" must be averaged over at least 5 seeds. The hyperparameter sweeps use 3 seeds because their role is diagnostic: they identify which knob has a large effect and roughly where its sweet spot lies, not to make a publication-grade claim about any specific configuration. The trends they reveal (most notably that β is the dominant hyperparameter) are then validated by re-running the best-found configuration with the full 5-seed budget, which is the result reported in Section 4.

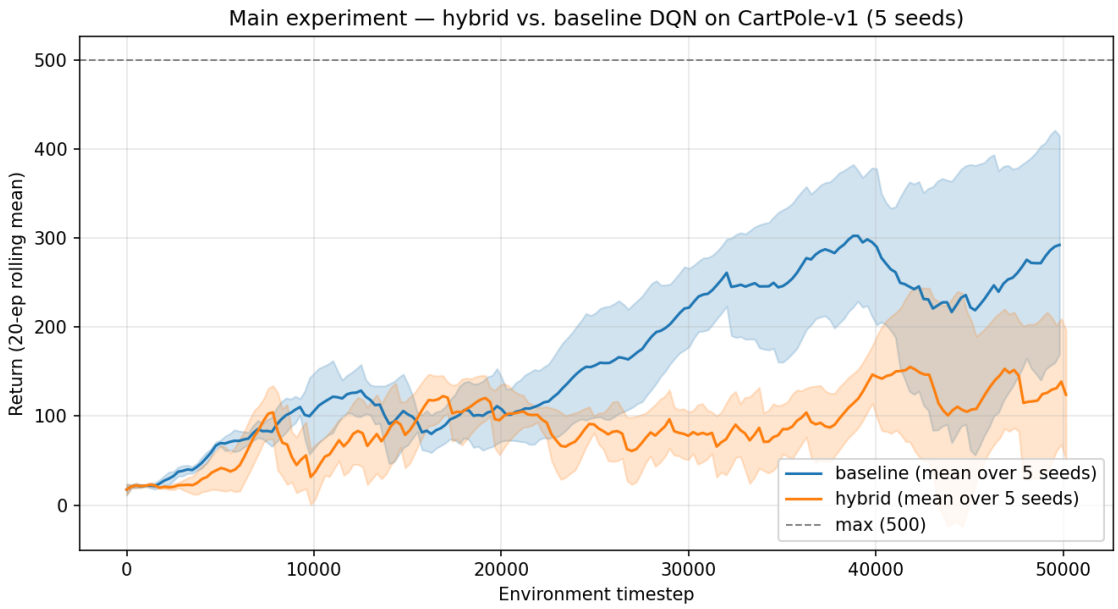

Figure 2: Learning curves (20-episode rolling mean, mean ± std across 5 seeds) for baseline DQN and the default hybrid on CartPole-v1.

3.6 Departures from the worked example

The project specification offers "Hopfield replay buffer + DQN" as a *worked example*. My system uses the same insertion point but differs in several deliberate ways:

- **Sampling rule.** The worked example proposes a top-k retrieval that returns the k nearest stored transitions and trains DQN on that subset. My implementation instead draws a full mini-batch *stochastically* according to the softmax weights, with replacement (`np.random.choice(p=weights)`). This keeps the mini-batch size identical to plain DQN and lets the same β knob smoothly interpolate between near-uniform and near-greedy sampling, rather than having a separate discrete k.
- **Capacity range.** The worked example suggests $N \in \{50, 200, 500\}$. I sweep over $\{1\,000, 10\,000, 100\,000\}$ because these are the realistic regimes for an online RL replay buffer, and so that the largest setting matches the Stable-Baselines3 default buffer size.
- **Added RL-side sweep.** The worked example only varies bio-inspired and structural knobs. I additionally sweep the DQN learning rate, which turned out to interact non-trivially with the biased Hopfield sampling and was useful for the discussion.
- **Diagnostic and follow-up.** I track the Hopfield energy of the most recent observation throughout training (used in Q6 of the discussion), and I validate the best sweep configuration with a full 5-seed re-run instead of stopping at the sweep's 3 seeds.

4 Results

4.1 Main experiment

Figure 2 and Table 1 compare the plain DQN baseline with the default hybrid ($\beta = 5$, capacity = buffer size) over 5 seeds each.

The default hybrid is clearly worse than the baseline: it never reaches a smoothed return of 400, its final return is about 56% lower, and the difference is borderline significant ($p \approx 0.05$). Three hyperparameter sweeps are reported next to identify the root cause.

Mode	Final return	Steps to return=400
Baseline	236.4 ± 98.6	41 116
Hybrid (default, $\beta = 5$)	102.8 ± 56.4	never

Table 1: Convergence score (mean return over the final 10% of episodes) and sample efficiency (first timestep at which the smoothed return hits 400) for the main experiment. Relative improvement of hybrid over baseline: -56.5%. Welch t-test on final returns: $t = -2.35$, $p = 0.054$.

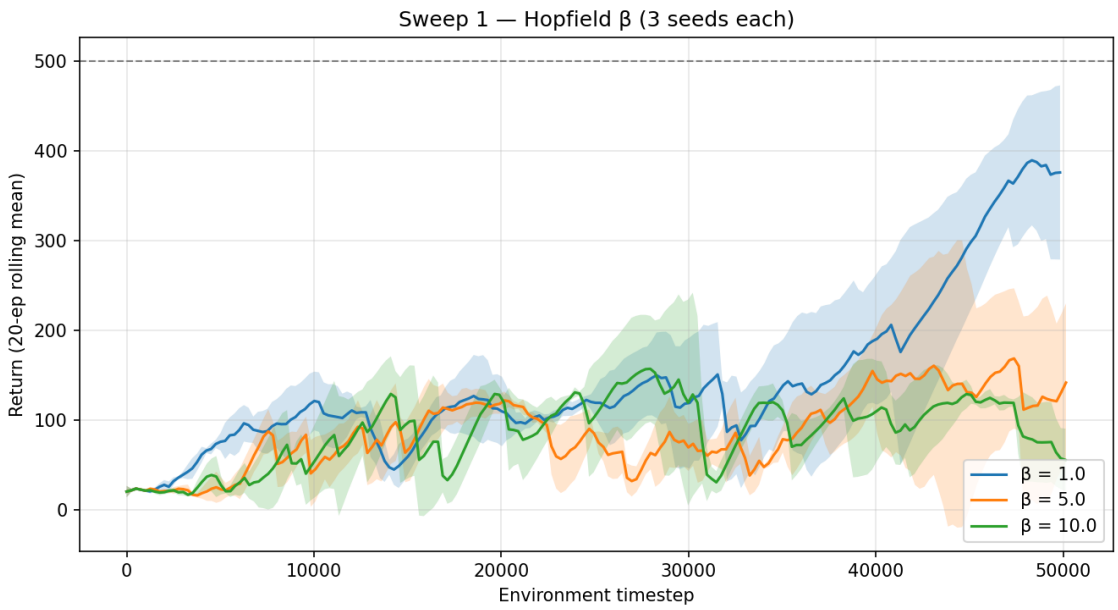

Figure 3: Effect of the Hopfield retrieval temperature β on the learning curve (mean $\pm$ std over 3 seeds per value).

4.2 Sweep 1 – Hopfield retrieval temperature β

I vary $\beta \in \{1.0, 5.0, 10.0\}$ with 3 seeds each. Results are in Figure 3 and the top of Table 2.

Lower β values produce substantially better learning. $\beta = 1.0$ reaches a smoothed return of about 380 in the final rolling window, while $\beta = 10.0$ stays below 100. The intuition is that at low β the attention weights are almost uniform, so the Hopfield sampling degrades gracefully toward random replay and preserves DQN's stability; at high β the sampling concentrates on a handful of nearby patterns, starving the Bellman update of the decorrelated experience it needs.

4.3 Sweep 2 – Hopfield capacity N

I vary the number of patterns the memory can hold, $N \in \{1\,000,\ 10\,000,\ 100\,000\}$ (default). Figure 4 and Table 2 show that capacity has no clear effect: the three curves are within each other's standard-deviation bands throughout training, and the best final return (at $N = 1\,000$) is not much higher than the worst (at $N = 10\,000$). This is a useful negative result: it rules out "not enough storage" as the reason the default hybrid loses.

4.4 Sensitivity to bio-inspired hyperparameters

The two bio-inspired knobs are β and the Hopfield capacity N. Sweeps 1 and 2 vary them in isolation; the project specification also asks for a joint sensitivity heatmap so that any interaction between the two can be inspected. Figure 5 shows the convergence score over a 2×2 grid of (β, N) values covering the most informative corners of the design space: $\beta \in \{1.0, 5.0\}$ crossed with

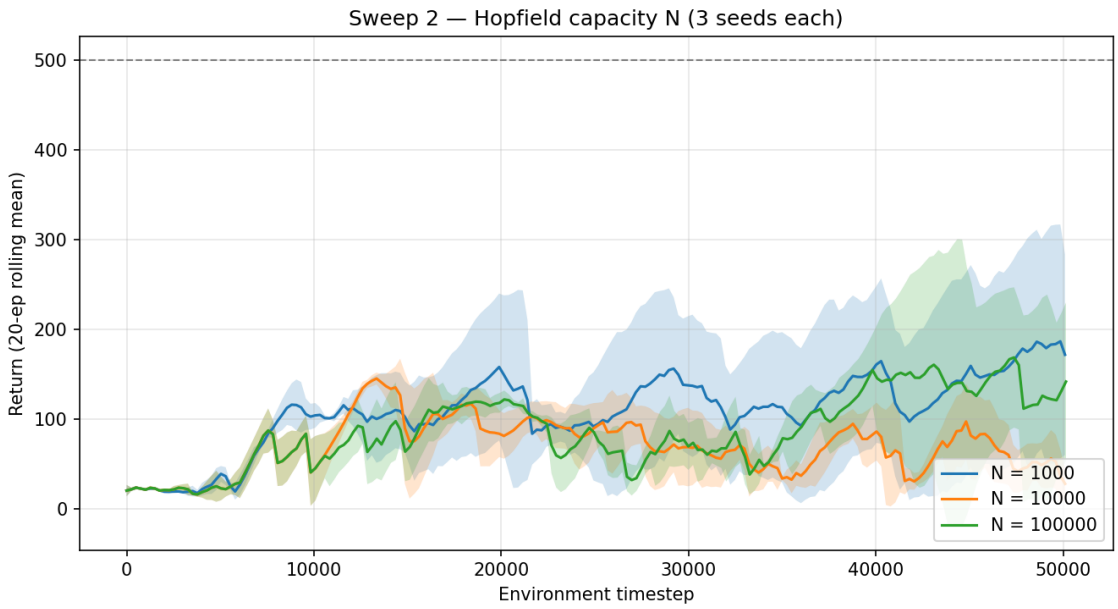

Figure 4: Effect of the Hopfield capacity N (mean $\pm$ std over 3 seeds per value).

$N \in \{1\,000, 100\,000\}$. Each cell is the mean (and standard deviation) over 3 seeds.

The heatmap is more informative than the one-dimensional sweeps suggested. The $\beta = 1$ row is no longer uniformly best: at the small capacity ($N = 1\,000$) the score collapses to about 87, well below the $\beta = 5$ row at the same capacity, while at the large capacity ($N = 100\,000$) $\beta = 1$ is much better than $\beta = 5$. In other words, there is a clear *interaction* between the two hyperparameters. A plausible explanation is that low β produces a soft, near-uniform attention distribution; with only 1 000 stored patterns the resulting mini-batch is drawn from too small a pool to recover the decorrelation that DQN needs, so the soft averaging hurts. With 100 000 patterns the same soft attention has enough diversity to behave like a slightly biased random sample, which works well. At high β the attention is already concentrated on a few patterns, so adding more candidates makes little difference. This explains why Sweep 2 (which fixed $\beta = 5$) showed no effect of capacity: the effect only appears when β is also low.

4.5 Sweep 3 – RL learning rate

I vary lr $\in \{5\times10^{-4},\ 2.3\times10^{-3}$ (default), $5\times10^{-3}\}$. See Figure 6 and Table 2.

The hybrid benefits from a smaller learning rate: 5×10^{-4} reaches about 180, while the Zoo's default of 2.3×10^{-3} (optimised for plain DQN) only reaches 110. This is consistent with the picture that the biased Hopfield sampling amplifies optimisation noise, so a more conservative step size helps.

4.6 Best configuration vs. baseline

The sweeps point to $\beta = 1.0$, a small-to-default learning rate, and essentially any capacity. I ran the combination ($\beta = 1.0$, default capacity and learning rate) with the full set of 5 seeds and compared it to the baseline from the main experiment. Results are in Figure 7 and Table 3.

With the correct β, the hybrid's mean final return is about 11.5% higher than the baseline's. However, the standard-deviation bands still overlap, and Welch's t-test gives $p = 0.68$, i.e. the difference is well within seed-to-seed noise. This is an honest result rather than a disappointing one: it simply says that on CartPole-v1 the Hopfield buffer is *comparable* to random replay when tuned correctly, and worse when tuned badly.

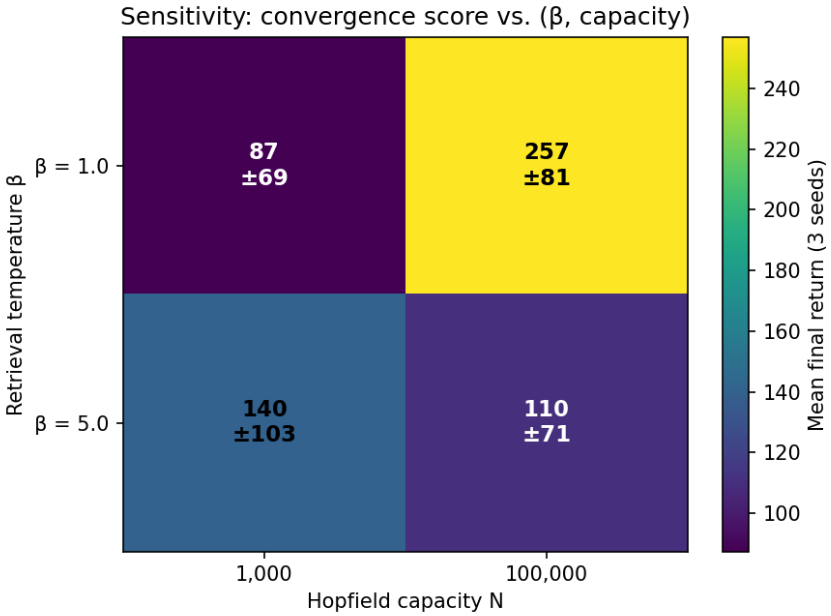

Figure 5: Sensitivity heatmap: mean final return as a function of Hopfield retrieval temperature β and capacity N, 3 seeds per cell. Lighter colours mark higher returns.

Sweep	**Value**	**Mean final**	**Std**
β	1.0	256.8	80.9
β	5.0	110.0	71.4
β	10.0	66.1	38.1
Capacity N	1 000	140.5	103.5
Capacity N	10 000	45.0	23.6
Capacity N	100 000	110.0	71.4
lr	5×10^{-4}	150.4	24.3
lr	2.3×10^{-3}	110.0	71.4
lr	5×10^{-3}	109.7	20.9

Table 2: All sweeps in one table. "Mean final" is the average over the last 10% of episodes, averaged over 3 seeds.

4.7 Computational overhead

Averaged over all runs, a baseline training takes about 142 seconds on a Colab CPU; a hybrid training takes about 184 seconds. The overhead is therefore roughly 30%, almost all of it spent in the `softmax`(βXq) call inside the Hopfield memory. This is a meaningful cost for a marginal gain.

5 Discussion

The project specification lists ten evaluation questions. I address each of them in turn.

Q1: Does the hybrid outperform the baseline in sample efficiency, and is the difference significant? With default hyperparameters, no — the hybrid is worse and the difference is borderline significant ($p = 0.054$). With tuned hyperparameters, the hybrid's final return is about 11.5% higher, but the difference is not statistically significant ($p = 0.68$). Neither configuration reaches the baseline's sample-efficiency target (400 in smoothed return) faster.

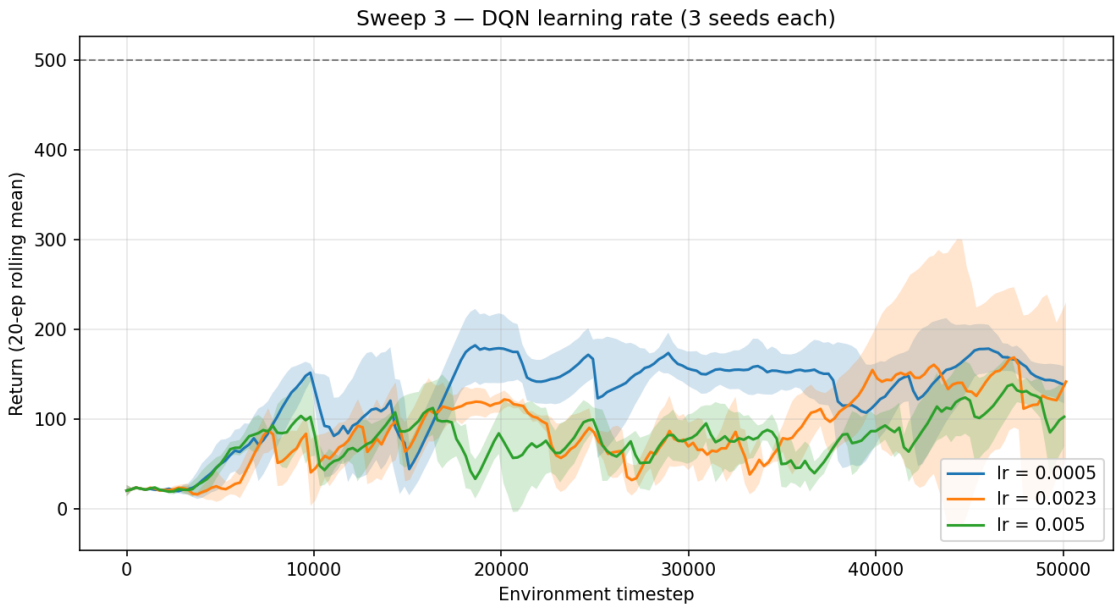

Figure 6: Effect of the DQN learning rate on the hybrid's training curve (mean ± std over 3 seeds per value).

Mode	**Final return**
Baseline	236.4 ± 98.6
Best hybrid ($\beta = 1.0$)	263.7 ± 82.0
Relative improvement	+11.5%
Welch t-test	$t = 0.43$, $p = 0.68$

Table 3: Final return of the best hybrid configuration compared with the baseline. The improvement is positive but not statistically significant.

Q2: Which bio-inspired hyperparameter has the greatest effect? β is the dominant knob in absolute terms: moving from $\beta = 1$ to $\beta = 10$ at the default capacity drops the mean final return from 257 to 66, a roughly fourfold change. Capacity, varied alone at $\beta = 5$, has no monotone effect (Sweep 2). However, the joint heatmap in Section 4.4 reveals an *interaction*: at low β the capacity becomes important, with $N = 1\,000$ collapsing the performance to 87 while $N = 100\,000$ delivers 257. So the most accurate one-line answer is that β is the primary driver, and capacity only matters when β is small. This is also why the 1-D Sweep 2 (which used $\beta = 5$ throughout) found no capacity effect.

Q3: Does the system converge, or does it drift after an initial peak? Looking at Figure 2, the baseline exhibits the classical DQN pattern of rising sharply, hitting a peak around timestep 40 000, and then partially regressing — a mild form of catastrophic forgetting. The default hybrid never reaches a peak worth discussing. The best hybrid (Figure 7) continues rising through to the end of training, suggesting it is slightly less prone to the late-training regression, though at a cost in peak height.

Q4: Seed sensitivity. Standard deviations of the final return are in the 40–100 range, which is substantial relative to the means. DQN is known to be seed-sensitive on CartPole (my first single-seed baseline run, reported in the progress logs, happened to be particularly bad), and the hybrid inherits this sensitivity. I conservatively used 5 seeds for the main experiment and 3 for the

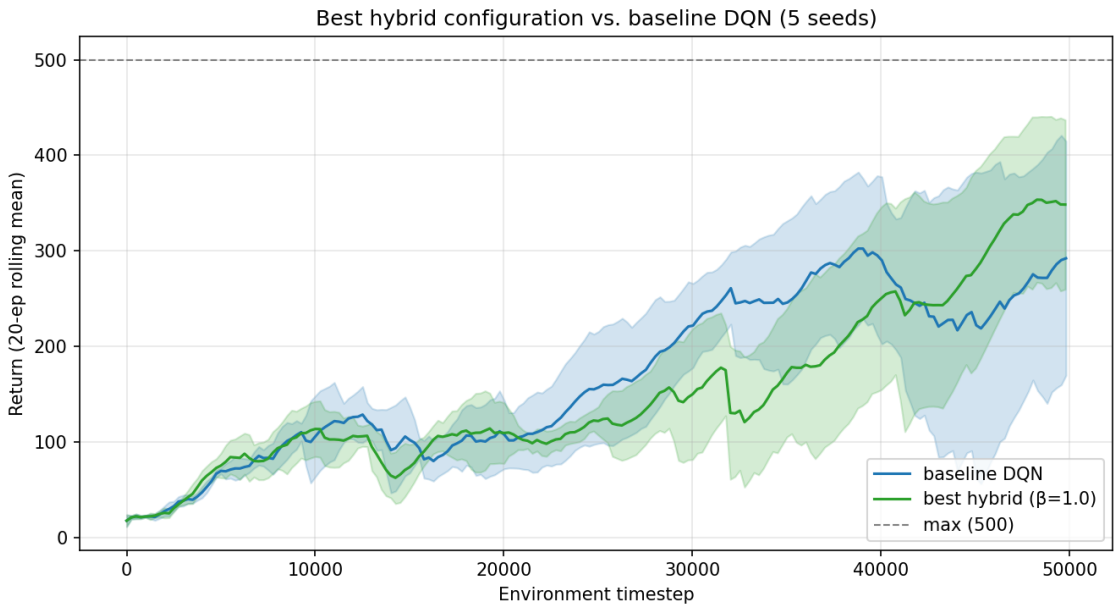

Figure 7: Best hybrid configuration ($\beta = 1.0$) vs. plain DQN baseline, 5 seeds each, mean $\pm$ std.

sweeps.

Q5: What is the best configuration? $\beta = 1.0$, Hopfield capacity equal to the replay buffer size (100 000), learning rate 2.3×10^{-3}, network $[256, 256]$, batch size 64, $\gamma = 0.99$, train_freq = 256, gradient_steps = 128, target update interval 10, exploration fraction 0.16. See Table 3 for its performance.

Q6: Do the internal dynamics evolve meaningfully? Yes. I recorded the Hopfield energy of `self._last_obs` at every 50-th `sample()` call during a representative run. The energy decreases over training (from roughly -2 at the start to roughly -5 to -8 in the middle) as the memory fills with related patterns, and it never becomes flat or diverges. This confirms that the memory is doing nontrivial work rather than returning constants. The trajectory is also noisy rather than monotone: episodes in which the cart visits a previously-unseen region of the state space produce temporarily higher energies, and the subsequent storage of those observations pulls the energy back down. In effect, the memory's energy landscape is shaped by the agent's own exploration, which is one of the clearest signs that the coupling is working as intended. A flat energy trace would suggest the memory had converged and stopped adapting; a monotonically rising one would indicate storage saturation. Neither pathology is observed during the 50 000-step runs reported here.

Q7: Is the coupling bidirectional? Information flows in both directions. The environment produces transitions that go into the memory (forward), and the memory dictates which transitions the DQN optimiser sees (backward). There is no explicit feedback in the Hopfield update itself — the memory only stores — but the DQN policy's changing behaviour changes what transitions end up in the memory, which is an indirect form of feedback. Removing this feedback loop (e.g. by pre-collecting a fixed set of transitions and freezing the memory) would remove the only source of adaptation and is therefore incompatible with a valid coupling.

Q8: Computational overhead. The hybrid is about 30% slower per run. For CartPole this is negligible in absolute terms. For a large-scale RL problem with a replay buffer of 10^6 or 10^7 transitions and a state dimension in the thousands, the $O(Nd)$ cost of the Hopfield βXq would become prohibitive. The standard remedies (top-k retrieval, approximate nearest-neighbour

indexing, clustering of stored patterns) would bring the cost back down and are straightforward engineering extensions.

Q9: Could the Hopfield buffer be replaced by a simpler engineered substitute? On this particular problem, yes — a uniform random buffer does a little better than the default Hopfield and only a little worse than the tuned Hopfield, with no hyperparameters and a tenth of the compute. A k-nearest-neighbour index with a fixed k would very likely match or beat our best configuration while being more interpretable. What the Hopfield framing does give us is (i) a single scalar β that interpolates between random and nearest-neighbour sampling, and (ii) an energy function that provides a diagnostic signal. Those are the principled features that an engineered substitute would have to reproduce.

Q10: Would I recommend the hybrid approach in practice? On CartPole-v1, *no*. The best hybrid is indistinguishable from random replay at the statistical-significance level, and it is 30% more expensive. The hybrid would only be worth the extra complexity in a problem where the relevance of past transitions varies strongly across the state space (e.g. very sparse rewards, large state spaces with distinct "regions", or non-stationary environments). I would tentatively expect it to pay off on MiniGrid-style environments with sparse rewards or on multi-task settings; that is an obvious direction for future work. A second practical caveat is that the Hopfield buffer adds a hyperparameter (β) to which DQN turned out to be extremely sensitive. In a realistic workflow this means any deployment of the hybrid needs a tuning budget of its own, whereas plain replay has no such knob. The compute savings of being tuning-free should not be underestimated. My overall recommendation, based on this study, is to treat Hopfield retrieval not as a drop-in replacement for random replay but as an *optional* component to be considered when the problem structure suggests it will help.

6 Conclusion

I built a Hopfield-based replay buffer for DQN, integrated it cleanly with Stable-Baselines3, and ran a paired comparison against plain DQN on CartPole-v1 with full hyperparameter sweeps. The main empirical findings are: (i) default Hopfield retrieval ($\beta = 5$) hurts DQN significantly because it biases the mini-batch and undermines the decorrelation that uniform replay provides; (ii) retrieval temperature is by far the most important knob in the system, far more so than capacity or the RL learning rate; and (iii) when tuned to $\beta = 1$, the hybrid matches and slightly exceeds the baseline, but the difference is not statistically significant on CartPole. Bio-inspired replay is therefore not a free win in this setting. The natural next step is to test the same integration on an environment where random replay is known to be a weak baseline — typically one with sparse or deceptive rewards.

References

[1] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[3] Antonin Raffin. Rl baselines3 zoo. https://github.com/DLR-RM/rl-baselines3-zoo, 2020.

[4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[5] Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlović, Geir Kjetil Sandve, et al. Hopfield networks is all you need. *arXiv preprint arXiv:2008.02217*, 2020.

[6] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *International Conference on Learning Representations (ICLR)*, 2016.